MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Daniel Moisei FAF-241

Report

Laboratory Work No. 3

Operating Systems

Topic: The Command Line: Shell, Pipes and Text Filters

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

CONTENTS

Introduction **3**

1 Move Fast and Find Help **4**

1.1 Tab completion and command history 4

1.2 Built-ins, external commands, and PATH 4

1.3 Help without a browser 5

2 What the Shell Does to a Command Line **6**

2.1 Test files 6

2.2 Predictions and observed results 6

3 Redirection, Chaining, and Jobs **9**

3.1 Standard output and standard error 9

3.2 Chaining operators 10

3.3 Background job control 10

4 Questions Answered with One-Liners **12**

4.1 Q1: Number of failed requests 12

4.2 Q2: Distinct requested pages 12

4.3 Q3: Requests per page 12

4.4 Q4: Users with the most failures 12

4.5 Q5: Requests made by user3 13

4.6 Q6: Last three failures 13

4.7 Q7: Login shells in /etc/passwd 13

4.8 Q8: Why long listing adds one line 14

4.9 Complete run and bonus 14

5 The First Bash Script **16**

5.1 Implementation 16

5.2 Successful execution 16

5.3 Error handling 17

Conclusion **18**

References **19**

INTRODUCTION

This laboratory work examined how a Unix shell interprets command lines before a program is executed, using Zsh interactively and Bash for the required scripts. The practical exercises covered command completion and history, built-in and external commands, wildcard expansion, quoting, redirection, command chaining, background jobs, pipelines, text filters, and a first Bash script.

The purpose of the work was to use the command line efficiently, predict shell expansion, control the standard input and output streams, and answer concrete questions about structured text by combining small filters. A generated access log containing 500 records was used as a reproducible data set.

The work was completed on a Linux system under the user account `moiseid`. Interactive exercises were performed in Zsh in the working directory `~/os-lab3`; the two deliverable scripts were executed explicitly with Bash. The screenshots record the local command aliases, `PATH`, installed login shells, and directory contents used for the observations below.

Preparation of the data set

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ]
    && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log ; wc -l access.log
```

The first three records had the expected five fields: date, time, user, result, and page. The final check printed `500 access.log`, confirming that the data set was complete.

1 MOVE FAST AND FIND HELP

1.1 Tab completion and command history

Typing `ls /usr/sh` and pressing Tab twice displayed the possible entries whose paths began with that prefix. Typing `cd /et` and pressing Tab once completed the unique path to `/etc/`. Thus, completion either inserts the unambiguous remainder or lists alternatives when several matches exist.

After three different commands had been executed, `Ctrl+R` started reverse incremental history search. Entering a distinctive fragment selected the earlier command containing it, and Enter executed that recalled command.

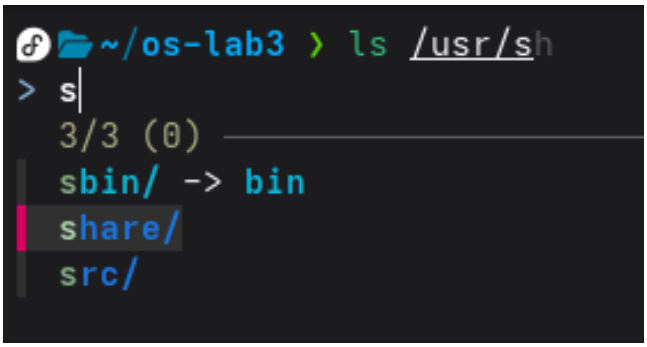


Figure 1.1: Tab completion alternatives for a partially typed path

1.2 Built-ins, external commands, and PATH

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog ; echo "exit code: $?"
```

Both `cd` and `echo` were reported as shell built-ins. In the interactive Zsh session, `type ls` showed that `ls` first resolved to the alias `ls -color=tty`; `which ls` displayed the alias and the underlying executable at `/usr/bin/ls`. The `cd` command must be a built-in because it changes the current directory of the running shell process. An external child process could change only its own working directory and would leave the parent shell unchanged after it exited.

During the recorded run, `PATH` contained six directories: `~/.local/bin`, `~/.local/share/zinit/pola` `/usr/local/bin`, `/usr/bin`, `/var/lib/snapd/snap/bin`, and `~/.dotnet/tools`. The shell searches them from left to right when a command contains no slash, and the underlying `ls` executable was found in `/usr/bin`.

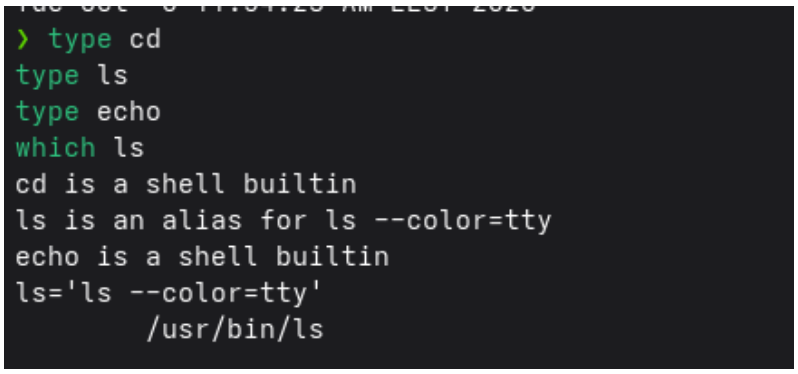


Figure 1.2: Identification of built-ins, the ls alias, and its executable

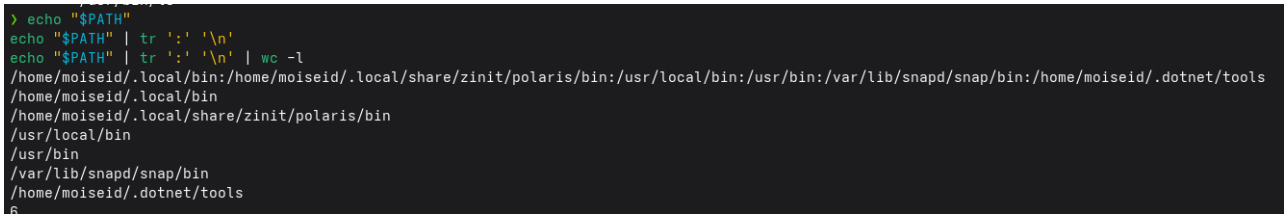


Figure 1.3: The six directories contained in PATH

The nonexistent command produced exit status `127`, the conventional shell status for "command not found." The `-t` option of `ls` sorts entries by modification time, newest first unless the order is reversed with another option.



Figure 1.4: Exit code returned by Zsh for a command that was not found

1.3 Help without a browser

```
man ls
ls --help | head
man -k "list directory"
help cd
```

The manual page provided the detailed interface of the external `ls` program, while `ls -help` displayed a concise option summary. The `man -k` command searched manual-page descriptions by keyword. Since `cd` is implemented by the shell, `help cd` displayed Bash's built-in documentation.

2 WHAT THE SHELL DOES TO A COMMAND LINE

2.1 Test files

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

The directory contained two C source names, three ordinary text-file names, and one filename containing a space. This controlled set made each expansion result predictable.

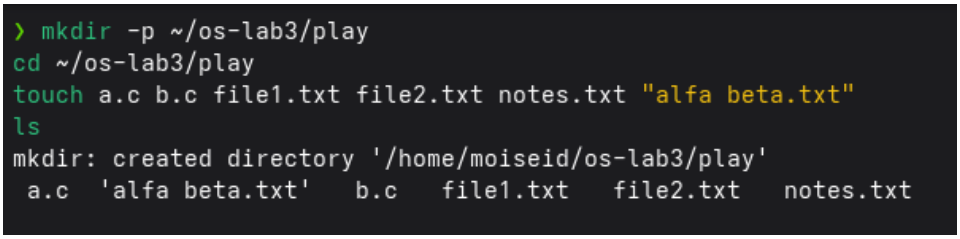


Figure 2.1: Files created in the play directory

2.2 Predictions and observed results

Table 1 records the predictions made before execution and the output observed afterwards. The value of the home directory and the weekday are specific to the execution environment.

Table 1: Predictions and results of shell expansion

Command	Prediction	Real output	Explanation
`echo *.c`	`a.c b.c`	`a.c b.c`	The shell expanded the matching filenames before running `echo`.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	The question mark matched exactly one character.
`echo [a-b]*`	Names beginning with a or b	`a.c alfa beta.txt b.c`	The bracket expression matched an initial `a` or `b`.
`echo *.xyz`	Literal `*.xyz`	`*.xyz`	With the current shell settings, the unmatched glob remained unchanged.
`ls alfa beta.txt`	Two missing-name errors	Errors for `alfa` and `beta.txt`	The unquoted space separated two arguments.
`ls "alfa beta.txt"`	The single filename	`alfa beta.txt`	Double quotes preserved the space inside one argument.
`echo '$HOME' "$HOME" \$HOME`	Literal, expanded, literal	`$HOME /home/moiseid $HOME`	Single quotes and the backslash prevented parameter expansion; double quotes allowed it.

Command	Prediction	Real output	Explanation
`echo {1..5}; echo $((7 * 6))`	`1 2 3 4 5` and `42`	`1 2 3 4 5` and `42`	The shell performed brace expansion and arithmetic expansion.
`echo "today is $(date +%A), I am $(whoami)"`	Weekday and current user	`today is Tuesday, I am moiseid`	Command substitution inserted the output of both commands.
`touch log{1..3}.txt && ls log*`	Three log files	`log1.txt log2.txt log3.txt`	Brace expansion created three names and the glob selected them.

```
❯ echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
echo {1..5}
echo $((7 * 6))
echo "today is $(date +%A), I am $(whoami)"
touch log{1..3}.txt
ls log*
a.c b.c
file1.txt file2.txt
a.c alfa beta.txt b.c
*.xyz
1 2 3 4 5
42
today is Tuesday, I am moiseid
log1.txt  log2.txt  log3.txt
```

Figure 2.2: Observed wildcard, brace, arithmetic, and command-substitution results

The `echo` command itself did not inspect the directory. The shell performed pathname expansion first and passed the resulting names to `echo` as arguments. The unquoted command `ls alfa beta.txt` failed because word splitting produced two names that did not exist. It can be fixed either by quoting the name, `ls "alfa beta.txt"`, or by escaping the space, `ls alfa\ beta.txt`.

```
❯ ls alfa beta.txt
ls "alfa beta.txt"
ls alfa\ beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
'alfa beta.txt'
'alfa beta.txt'
```

Figure 2.3: Failure caused by an unquoted space and two corrected forms

Single quotes preserve every character literally until the closing quote. Double quotes also keep spaces inside one argument, but still allow parameter expansion, command substitution, and arithmetic expansion. A backslash protects the next character. When `ls` displays `'alfa beta.txt'` with quotes, the quotes are presentation added to make the space unambiguous; they are not part of the filename.

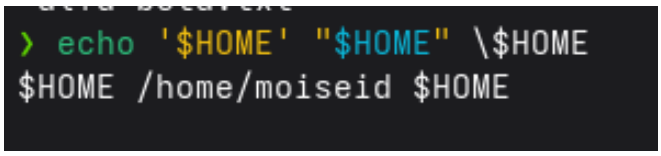


Figure 2.4: Single quotes, double quotes, and backslash quoting

3 REDIRECTION, CHAINING, AND JOBS

3.1 Standard output and standard error

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
```

In the first command, file descriptor 1, standard output, was redirected to `out.txt`; therefore that file received the `/etc` listing, which contained 311 lines on this system. File descriptor 2, standard error, was redirected independently to `err.txt`, which received the diagnostic for `/nope`. The > operator replaced a file's previous content, whereas » appended to it.

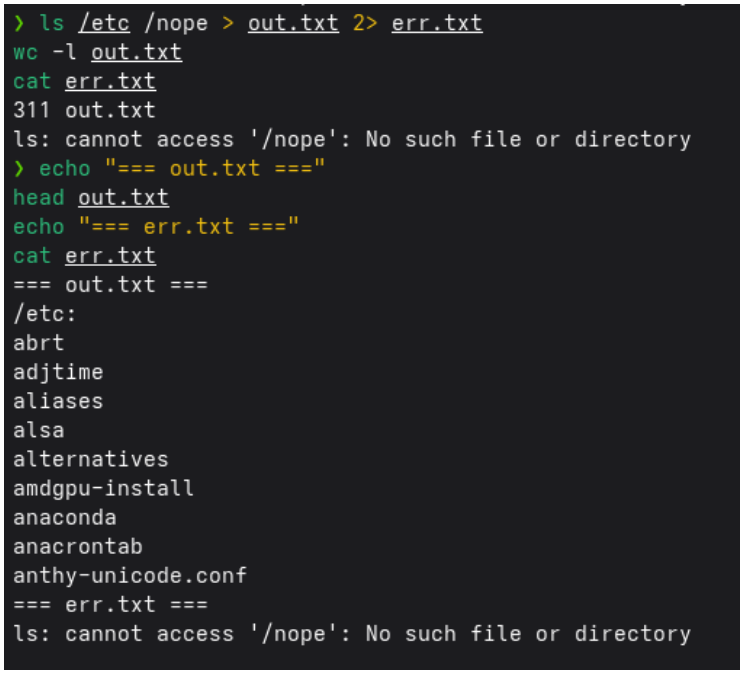


Figure 3.1: Standard output and standard error redirected to separate files

The order of redirections changed their meaning. In `> all.txt 2>&1`, standard output was first attached to the file and standard error was then duplicated from the new standard output, so both streams entered `all.txt`. In `2>&1 > all.txt`, standard error was first duplicated from the terminal, after which only standard output moved to the file. The error consequently remained visible in the terminal.

```
❯ ls /etc /nope > all.txt 2>&1
echo "=== Terminal finished; file content ==="
tail -5 all.txt
=== Terminal finished; file content ===
zlogin
zlogout
zprofile
zshenv
zshrc
❯ ls /etc /nope 2>&1 > all.txt
echo "=== File content ==="
tail -5 all.txt
ls: cannot access '/nope': No such file or directory
=== File content ===
zlogin
zlogout
zprofile
zshenv
zshrc
```

Figure 3.2: Comparison of the two redirection orders

The command `wc -l < access.log` received data through standard input and therefore knew no filename to print; it displayed only `500`. In contrast, `wc -l access.log` opened a named argument and included that name after the count.

3.2 Chaining operators

```
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
```

The semicolon runs the following command regardless of the preceding exit status. The && operator runs its right-hand command only when the left-hand command succeeds, while || runs its right-hand command only when the left-hand command fails. Accordingly, `ls /nope` triggered the fallback message with exit code 2. The chained directory changes succeeded and printed `/home/moiseid/os-lab3/tmp`; the two commands separated by a semicolon then printed `first` and `second` unconditionally.

```
❯ ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)"
cd ~/os-lab3
echo first ; echo second
failed with code 2
mkdir: created directory 'tmp'
I am in /home/moiseid/os-lab3/tmp
first
second
```

Figure 3.3: Behavior of the semicolon, AND, and OR command operators

3.3 Background job control

```
sleep 100 &
jobs ; kill %1 ; jobs
```

The ampersand started `sleep` asynchronously and returned control to the prompt. The `jobs` built-in

listed it as job 1 with PID 91311, `kill %1` sent a termination signal to that job, and the next status display showed that it had terminated.

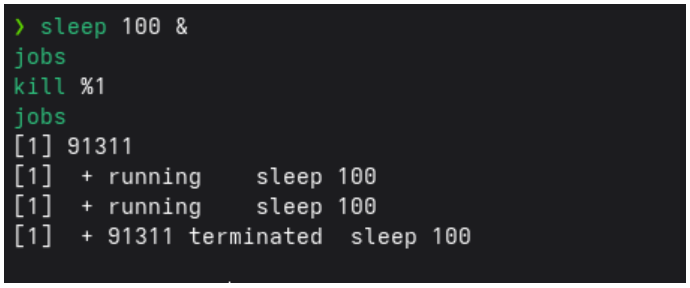


Figure 3.4: Starting, listing, and stopping a background job

4 QUESTIONS ANSWERED WITH ONE-LINERS

Each pipeline below was saved in `oneliners.sh`. The artificial access log is deterministic, so its counts can be checked independently: the page totals sum to 500 and the failure counts per user sum to 100.

4.1 Q1: Number of failed requests

```
grep -c FAIL access.log
```

The command printed `100`; one of every five generated requests had result `FAIL`.

```
❯ grep -c FAIL access.log
100
```

Figure 4.1: Number of failed requests

4.2 Q2: Distinct requested pages

```
cut -d' ' -f5 access.log | sort -u | awk 'BEGIN { print "Requested pages:" } { pages[
    NR]=$0 } END { for (i=1; i<=NR; i++) print pages[i]; print "Number of different
    pages: " NR }'
```

There were four pages: `/page0`, `/page1`, `/page2`, and `/page3`.

```
❯ cut -d' ' -f5 access.log | sort -u
cut -d' ' -f5 access.log | sort -u | wc -l
/page0
/page1
/page2
/page3
4
```

Figure 4.2: Number and names of distinct requested pages

4.3 Q3: Requests per page

```
cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
```

Each of the four pages received exactly 125 requests; their sum was 500.

```
❯ cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
    125 /page3
    125 /page2
    125 /page1
    125 /page0
```

Figure 4.3: Request count for each page

4.4 Q4: Users with the most failures

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR == 1 {
    maximum=$1 } $1 == maximum'
```

Users `user3` and `user5` tied for the maximum with 15 failures each. Every other user had 14 failures, so all per-user counts summed to 100.

```
❯ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
```

Figure 4.4: Users with the largest number of failed requests

4.5 Q5: Requests made by user3

```
printf 'user3 requests: '; grep -c ' user3 ' access.log; printf 'user3 failed requests
    : '; grep ' user3 ' access.log | grep -c FAIL
```

The result was 72 requests made by `user3`, of which 15 failed.

```
❯ grep -c ' user3 ' access.log
grep ' user3 ' access.log | grep -c FAIL
72
15
```

Figure 4.5: Total and failed requests for user3

4.6 Q6: Last three failures

```
grep FAIL access.log | tail -3 | cut -d' ' -f2-3
```

The last three matching records, reduced to time and user, were `10:10 user0`, `10:15 user5`, and `10:20 user3`.

```
❯ grep FAIL access.log | tail -3 | cut -d' ' -f2-3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6: Time and user of the last three failed requests

4.7 Q7: Login shells in /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

The seventh colon-separated field was extracted from every account and equal shell paths were counted. The system contained 21 accounts using `/usr/bin/nologin`, 16 using `/sbin/nologin`, 14 using `/usr/sbin/nologin`, and 2 using `/bin/bash`. One account each used `/usr/bin/zsh`, `/sbin/shutdown`, `/sbin/halt`, and `/bin/sync`.

```
❯ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     21 /usr/bin/nologin
     16 /sbin/nologin
     14 /usr/sbin/nologin
      2 /bin/bash
      1 /usr/bin/zsh
      1 /sbin/shutdown
      1 /sbin/halt
      1 /bin/sync
```

Figure 4.7: Login shells and the number of accounts using each shell

4.8 Q8: Why long listing adds one line

```
printf 'ls /etc count: '; ls /etc | wc -l; printf 'ls -l /etc count: '; ls -l /etc |
    wc -l
```

The prediction was that the long-listing pipeline would contain exactly one extra line. Ordinary `ls /etc` produced 310 lines. With `-l`, `ls` added a leading `total` line that reports the filesystem blocks used by the listed entries, so the second pipeline produced 311 lines. The pipe counted this header as well, producing the predicted difference of one.

```
❯ ls /etc | wc -l
ls -l /etc | wc -l
310
311
```

Figure 4.8: Comparison of line counts for ordinary and long directory listings

4.9 Complete run and bonus

```
bash oneliners.sh
```

The complete script produced all answers consecutively. The bonus regular expression selected failed records whose minute ended in zero. It matched 50 lines in this generated data set.

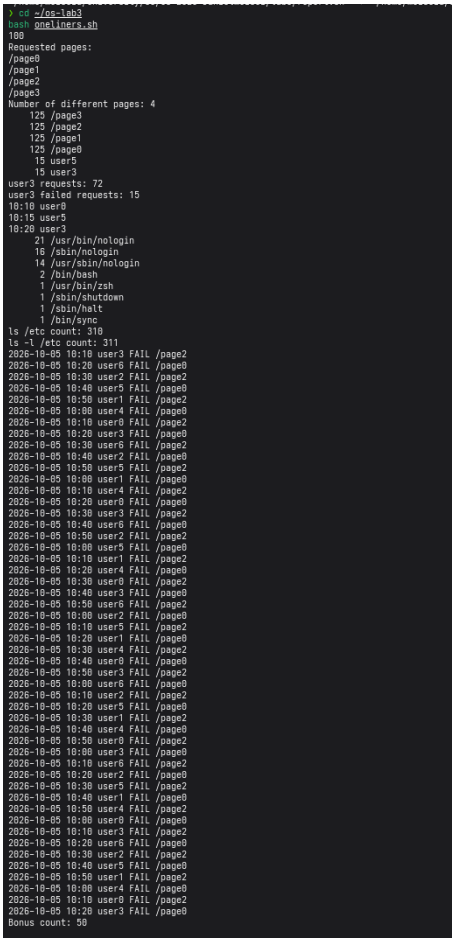


Figure 4.9: Full execution of oneliners.sh

5 THE FIRST BASH SCRIPT

5.1 Implementation

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for user in user{0..6}; do
    count=$(grep FAIL "$log" | grep -c " $user ")
    echo "$user: $count failures"
done
```

The loop expanded the sequence from `user0` through `user6`. For each user it filtered the failed records, counted the records containing that exact space-delimited username, and printed the result.

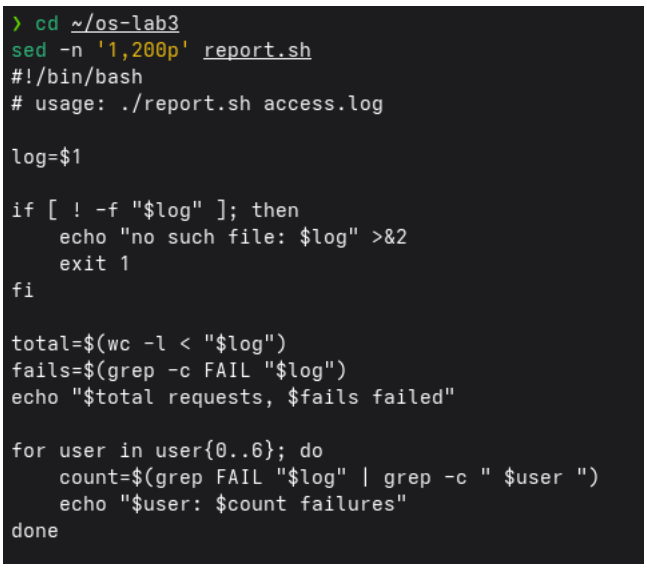


Figure 5.1: Completed contents of report.sh

5.2 Successful execution

```
chmod +x report.sh
./report.sh access.log
```

The script reported `500 requests, 100 failed`. Users 0, 1, 2, 4, and 6 each had 14 failures, while users 3 and 5 each had 15.



Figure 5.2: Execution of report.sh with the generated access log

The first line, `#!/bin/bash`, is the shebang. When the file is executed directly, it tells the kernel to use `/bin/bash` as its interpreter. Before `chmod +x report.sh`, direct execution fails with a permission-denied error because the executable permission bit is absent, although `bash report.sh access.log` can still ask Bash to read the file explicitly.

The command is typed as `./report.sh` because the current directory is normally not present in `PATH`. The explicit `./` path selects the file in the current directory and avoids accidentally executing a different program with the same name from a searched directory.

5.3 Error handling

```
./report.sh nope.log ; echo "exit code: $?"
```

The diagnostic was written with >&2 so that it used the standard error stream and could be redirected independently from normal report output. The explicit `exit 1` returned a nonzero status to communicate failure to the calling shell or another script. The following `echo` consequently displayed exit code 1.

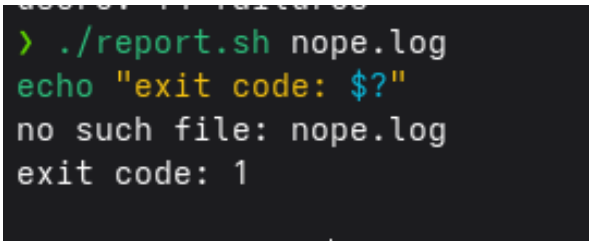


Figure 5.3: Error handling and exit status for a missing input file

CONCLUSION

This laboratory demonstrated that the shell performs substantial processing before a program receives its arguments. Tab completion and history search accelerated interactive use, while `type`, `which`, `PATH`, manual pages, and built-in help explained how commands are located and documented. The expansion exercises showed that the shell, rather than `echo`, resolves wildcard patterns and that quoting determines whether spaces and special characters remain literal.

Redirection made the distinction between standard output and standard error visible, including the left-to-right significance of redirection order. The chaining operators used exit statuses to control later commands, and job control allowed a process to run asynchronously and then be terminated from the shell.

Finally, pipelines composed `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, `tail`, `tr`, and `awk` into reproducible answers about the access log. The completed script added argument handling, input validation, error reporting, variables, a loop, and meaningful exit status. Together, these exercises established a practical basis for efficient Linux command-line work and small automation tasks.

REFERENCES

[1] Operating Systems Laboratory Work No. 3. *The Command Line: Shell, Pipes and Text Filters*. FCIM / FAF / UTM, 2026–2027.

[2] GNU Bash manual and Linux manual pages consulted through `help` and `man` during the laboratory.